UNIT 2: OPERATORS

Prachi Surlakar
Assistant Professor
Computer Engineering Department
Goa College of Engineering

Unit - 2	**Introduction:** Features of Python, execution of Python programs, Python virtual machines, memory management, garbage collection, comparison between C and Python. **Data Types:** Comments, docstrings, built-in data types, strings, sets, literals, user-defined data types, constants, identifiers, reserved words and naming conventions in python. **Operators:** Arithmetic, assignment, unary, relational, logical, Boolean, bitwise, membership, identity operators, operator precedence and associativity. **Control statements:** if, if-else, if-elif else, while, for, nested loops, break, continue, pass, assert and return statements	**12**

2. Dr. R. Nageswara Rao; "Core Python Programming", Dreamtech press, Third edition, 2018.

OPERATORS IN PYTHON

OPERATOR: Operator is a symbol that performs an operation.

Example: a + b

'+' : operator a, b : operands

- **Unary operator**: Operator acts on a single variable.
- **Binary operator**: Operator works on two variables.
- **Ternary operator**: Operator works on three variables.

We can classify operators as:

1. Arithmetic operators
2. Assignment operators
3. Unary minus operators
4. Relational operators
5. Logical operators
6. Boolean operators
7. Bitwise operators
8. Membership operators
9. Identity operators

1. ARITHMETIC OPERATORS

Also called as binary operators.
Assume a=13, b=5

Table 4.1: Arithmetic Operators in Python

Operator	Meaning	Example	Result
+	Addition operator. Adds two values.	a+b	18
-	Subtraction operator. Subtracts one value from another.	a-b	8
*	Multiplication operator. Multiplies values on either side of the operator.	a*b	65
/	Division operator. Divides left operand by the right operand.	a/b	2.6
%	Modulus operator. Gives remainder of division.	a%b	3
**	Exponent operator. Calculates exponential power value. a ** b gives the value of a to the power of b.	a**b	371293
//	Integer division. This is also called Floor division. Performs division and gives only integer quotient.	a//b	2

1. ARITHMETIC OPERATORS

Precedence:

1. Parentheses
2. Exponentiation
3. Multiplication, division, modulus, floor division
4. Addition, subtraction
5. Assignment operation

Example:

d=(x+y) *z** a//b +c

(Assume x=1, y=2, z=3, a=2, b=2, c=3)

d= (1+2) *3 ** 2 // 2+ 3

d= 3 *3 ** 2 // 2+ 3

d= 3 *9// 2+ 3

d= 27// 2+ 3

d= 13+ 3

d=16

Example:

```
d=3**2//2+3
print(d)
```

Output:7

d=3**2//2+3

d=9//2+3

d=4+3

d= 7

1. ARITHMETIC OPERATORS

Using python interpreter as calculator

```
>>> 13+5
18
>>> 13-5
8
>>> 13*5
65
>>> 13/5
2.6
>>> 13%5
3
>>> 13**5
371293
>>> 13//5
2
>>> d = (1+2)*3**2//2+3
>>> print(d)
16
```

2. ASSIGNMENT OPERATORS

- Store right side value into a left side variable.
- Can also be used to perform simple arithmetic operations like addition, subtraction etc.
- Assume x=20 and y=10 z=5

Operator	Example	Meaning	Result
=	z = x+y	Assignment operator. Stores right side value into left side variable, i.e. x+y is stored into z.	z = 30
+=	z+=x	Addition assignment operator. Adds right operand to the left operand and stores the result into left operand, i.e. z = z+x.	z = 25
-=	z-=x	Subtraction assignment operator. Subtracts right operand from left operand and stores the result into left operand, i.e. z = z-x.	z = -15
*=	z*=x	Multiplication assignment operator. Multiplies right operand with left operand and stores the result into left operand, i.e. z = z *x.	z = 100

2. ASSIGNMENT OPERATORS

- Store right side value into a left side variable.
- Can also be used to perform simple arithmetic operations like addition, subtraction etc.
- Assume x=20 and y=10 , z=5

/=	z/=x	Division assignment operator. Divides left operand with right operand and stores the result into left operand, i.e. z = z/x.	z = 0.25
%=	z%=x	Modulus assignment operator. Divides left operand with right operand and stores the remainder into left operand, i.e. z = z%x.	z = 5
**=	z**=y	Exponentiation assignment operator. Performs power value and then stores the result into left operand, i.e. z = z**y.	z= 9765625
//=	z//=y	Floor division assignment operator. Performs floor division and then stores the result into left operand, i.e. z = z// y.	z = 0

2. ASSIGNMENT OPERATORS

It is possible to assign the same value to two variables in the same statement as:

```
a=b=1
print(a, b) # will display 1 1
```

Another example is where we can assign different values to two variables as:

```
a=1; b=2
print(a, b) # will display 1 2
```

The same can be done using the following statement:

```
a, b = 1, 2
print(a, b) # will display 1 2
```

Python **does not** have increment operator(++) and decrement operator(--)

3. UNARY MINUS OPERATOR

- Denoted by symbol minus(-)
- If the variable value is positive it is converted to negative and vice versa.

```
n=10
print(-n)    #prints -10
n=-10
print(-n)   #prints 10
```

4. RELATIONAL OPERATORS

These operators will return either true or false. Assume a=1 and b=2

Operator	Example	Meaning	Result
>	a>b	Greater than operator. If the value of left operand is greater than the value of right operand, it gives True else it gives False.	False
>=	a>=b	Greater than or equal operator. If the value of left operand is greater or equal than that of right operand, it gives True else False.	False
<	a<b	Less than operator. If the value of left operand is less than the value of right operand, it gives True else it gives False.	True
<=	a<=b	Less than or equal operator. If the value of left operand is lesser or equal than that of right operand, it gives True else False.	True
==	a==b	Equals operator. If the value of left operand is equal to the value of right operand, it gives True else False.	False
!=	a != b	Not equals operator. If the value of the left operand is not equal to the value of right operand, it returns True else it returns False.	True

4. RELATIONAL OPERATORS

- Mostly used to construct conditions.

```
a=1; b=2
if (a>b):
      print("Yes")
else:
      print("No")
```

OUTPUT: **No**

```
x=15
10<x<20   # displays True

10>=x<20 # displays False

10<x>20 # displays False
```

If we get **all true** then **only final result will be true**. If any comparisons yields false, then we get false as the final result.

```
1<2<3<4 # will give True

1<2>3<4 # will give False

4>2>=2>1 # will give True
```

5. LOGICAL OPERATORS

- Construct compound conditions.
- False:0
- True: any other number.

Consider x=1 and y=2

Table 4.4: Logical Operators

Operator	Example	Meaning	Result
And	x and y	And operator. If x is False, it returns x, otherwise it returns y.	2
Or	x or y	Or operator. If x is False, it returns y, otherwise it returns x.	1
Not	not x	Not operator. If x is False, it returns True, otherwise False	False

1 and 2 =2
0 and 2 =0
1 or 2 =1
0 or 2 = 2

not 2= False
not 0 =True

Note: use print to display the output.

5. LOGICAL OPERATORS

```
x = 100
y = 200
print(x and y)   # will display 200

print(x or y)   # will display 100

print (not x) # will display False
```

```
x=1;  y=2; z=3
if(x<y and y<z): print('Yes')
else: print('No')
```

OUTPUT: 'Yes'

When 'and' is used, total condition will be true only if both conditions are true.

```
if(x>y or y<z): print('Yes')
else: print('No')
```

OUTPUT: 'Yes'

When 'or' is used, if any one condition is true it will take total compound condition as true..

6.BOOLEAN OPERATORS

- **There are two 'bool' type literals. They are 'True' and 'False'**

Example : x=True y=False

Table 4.5: Boolean Operators

Operator	Example	Meaning	Result
and	x and y	Boolean and operator. If both x and y are True, then it returns True, otherwise False.	False
or	x or y	Boolean or operator. If either x or y is True, then it returns True, else False.	True
not	not x	Boolean not operator. If x is True, it returns False, else True.	False

6.BOOLEAN OPERATORS

```
>>> a = True
>>> b = False
>>> a and a
True
>>> a and b
False
>>> a and a
True
>>> a or a
True
>>> a or b
True
>>> b or b
False
>>> not a
False
>>> not b
True
```

7. BITWISE OPERATORS

- **Bitwise operators act** on **individual bits (0 and 1)** of the operands.
- We can **use** bitwise operators **directly** on **binary numbers** or on **integers** also.
- When we **use these operators on integers**, these **numbers are converted into bits**(binary number system) and then bitwise **operators** act upon those **bits**.
- **Results** are in the form of **integers**
- **Decimal number system : consists numbers from 0 to 9.**
- **Binary number system: there are only 2 digits (0 and 1)**
- **There are six types of bitwise operators:**

1. **Bitwise AND operator (&)**
2. **Bitwise OR operator (|)**
3. **Bitwise XOR operator (^)**
4. **Bitwise Left shift operator (<<)**
5. **Bitwise Right shift operator (>>)**
6. **Bitwise complement operator (~)**

7. BITWISE OPERATOR

BITWISE OPERATORS

- There are six types of bitwise operators:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise Left shift operator (<<)
5. Bitwise Right shift operator (>>)
6. Bitwise complement operator (~)

&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 **&** 14

First convert 13 to binary

2	13	1
	6	

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

First convert 13 to binary

2	13	1
2	6	0
	3	

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

First convert 13 to binary

2	13	1
2	6	0
2	3	1
	1	

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

First convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

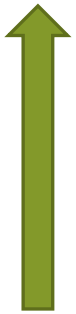
&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

First convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	



Write in 8 bits

13 = 0 0 0 0 1 1 0 1

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

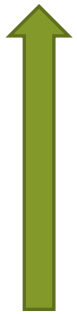
&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	



Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
	7	

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

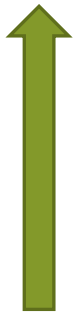
&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	



Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
2	7	1
	3	

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

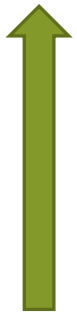
&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	



Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
2	7	1
2	3	1
	1	

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

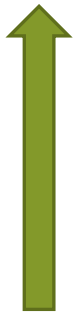
&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	



Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
2	7	1
2	3	1
2	1	1
	0	

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

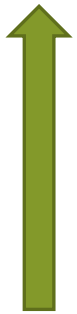
&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	

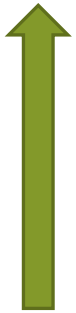


Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
2	7	1
2	3	1
2	1	1
	0	



7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

convert 13 to binary

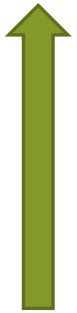
2	13	1
2	6	0
2	3	1
2	1	1
	0	

Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
2	7	1
2	3	1
2	1	1
	0	



Write in 8 bits

14 = 0 0 0 0 1 1 1 0

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

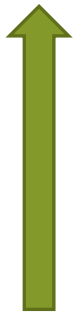
&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	

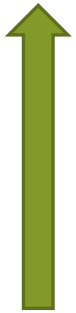


Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
2	7	1
2	3	1
2	1	1
	0	



Write in 8 bits

14 = 0 0 0 0 1 1 1 0

13 & 14

13 = 0 0 0 0 1 1 0 1

14 = 0 0 0 0 1 1 1 0

0 0 0 0 1 1 0 0

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

13 & 14

convert 13 to binary

Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

Write in 8 bits

14 = 0 0 0 0 1 1 1 0

13 & 14

13 = 0 0 0 0 1 1 0 1

14 = 0 0 0 0 1 1 1 0

0 0 0 0 1 1 0 0

0 0 0 0 1 1 0 0 → convert to decimal

0	0	0	0	1	1	0	0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=0 * 2^0 + 0 * 2^1 + 1 * 2^2 + 1 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$

$=0+0+4+8+0+0+0+0$

$=12$

13 & 14 = 12

7. BITWISE OPERATOR

BITWISE OPERATORS

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

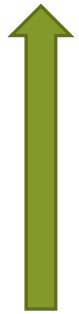
&(BITWISE AND) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying AND is 1 if both are 1 otherwise 0**

5 & 3

convert 5 to binary

2	5	1
2	2	0
2	1	1
	0	



Write in 8 bits

5 = 0 0 0 0 0 1 0 1

convert 3 to binary

2	3	1
2	1	1
	0	

Write in 8 bits

3 = 0 0 0 0 0 0 1 1

5 & 3

5 = 0 0 0 0 0 1 0 1

3 = 0 0 0 0 0 0 1 1

0 0 0 0 0 0 0 1

0 0 0 0 0 0 0 1 → convert to decimal

0	0	0	0	0	0	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=1 * 2^0 + 0 * 2^1 + 0 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$

=1+0+0+0+0+0+0+0

=1

5 & 3 = 1

7. BITWISE OPERATOR

BITWISE OPERATORS

BITWISE OR

- There are six types of bitwise operators:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise Left shift operator
5. Bitwise Right shift operator
6. Bitwise complement operator

A	B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

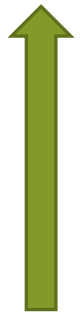
| (BITWISE OR) OPERATOR

- Every number will be stored in the form of bits(0s and 1s)
- **Result of applying OR is 1 if at least one bit value is '1' otherwise '0'**

13 | 14

convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	



Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
2	7	1
2	3	1
2	1	1
	0	

Write in 8 bits

14 = 0 0 0 0 1 1 1 0

13 | 14

13 = 0 0 0 0 1 1 0 1

14 = 0 0 0 0 1 1 1 0

0 0 0 0 1 1 1 1

0 0 0 0 1 1 1 1 → convert to decimal

0	0	0	0	1	1	1	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=1 * 2^0 + 1 * 2^1 + 1 * 2^2 + 1 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$

=1+2+4+8+0+0+0+0

=15

13 | 14 = 15

7. BITWISE OPERATOR

BITWISE OPERATORS

BITWISE XOR

- There are six types of bitwise operators:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise Left shift operator
5. Bitwise Right shift operator
6. Bitwise complement operator

A	B	A^B
0	0	0
0	1	1
1	0	1
1	1	0

^(BITWISE XOR) OPERATOR

- Result of applying XOR is 1 if ONLY one bit value is '1' otherwise '0'

13 ^ 14

convert 13 to binary

2	13	1
2	6	0
2	3	1
2	1	1
	0	

Write in 8 bits

13 = 0 0 0 0 1 1 0 1

convert 14 to binary

2	14	0
2	7	1
2	3	1
2	1	1
	0	

Write in 8 bits

14 = 0 0 0 0 1 1 1 0

13 ^ 14

13 = 0 0 0 0 1 1 0 1

14 = 0 0 0 0 1 1 1 0

0 0 0 0 0 0 1 1

0 0 0 0 0 0 1 1 → convert to decimal

0	0	0	0	0	0	1	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=1 * 2^0 + 1 * 2^1 + 0 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$

=1+2+0+0+0+0+0+0

=3

13 ^ 14 = 3

7. BITWISE OPERATOR

BITWISE OPERATORS

Write the output of the following

```
x=13
y=14
print('x and y is',x & y)
print('x or y is',x | y)
print('x xor y is',x ^ y)
```

OUTPUT:

```
x and y is 12
x or y is 15
x xor y is 3
```

7. BITWISE OPERATOR

BITWISE OPERATORS

- There are six types of bitwise operators:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise Left shift operator
5. Bitwise Right shift operator
6. Bitwise complement operator

<< Left shift operator

Shifts bit to the left
a=10, perform a<<1 and a<<2

Convert 10 to binary

2	10	0
2	5	1
2	2	0
2	1	1
	0	

a=10 = 0 0 0 0 1 0 1 0

a<<1

0 0 0 0 1 0 1 0

0 0 0 1 0 1 0 0

0	**0**	**0**	**1**	**0**	**1**	**0**	**0**
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=0 * 2^0 + 0 * 2^1 + 1 * 2^2 + 0 * 2^3 + 1 * 2^4$
$+0 * 2^5 + 0 * 2^6 + 0 * 2^7$

=0+0+4+0+16+0+0+0 =20

7. BITWISE OPERATOR

BITWISE OPERATORS

- There are six types of bitwise operators:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise Left shift operator
5. Bitwise Right shift operator
6. Bitwise complement operator

<< Left shift operator

Shifts bit to the left
a=10, perform a<<1 and a<<2

Convert 10 to binary

2	10	0
2	5	1
2	2	0
2	1	1
	0	

a=10 = 0 0 0 0 1 0 1 0

a<<1

~~0~~ 0 0 0 1 0 1 0
0 0 0 1 0 1 0 0

0	**0**	**0**	**1**	**0**	**1**	**0**	**0**
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=0 * 2^0 + 0 * 2^1 + 1 * 2^2 + 0 * 2^3 + 1 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$

$=0+0+4+0+16+0+0+0 = 20$

a<<2

~~0 0~~ 0 0 1 0 1 0
0 0 1 0 1 0 0 0

0	**0**	**1**	**0**	**1**	**0**	**0**	**0**
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=0 * 2^0 + 0 * 2^1 + 0 * 2^2 + 1 * 2^3 + 0 * 2^4 + 1 * 2^5 + 0 * 2^6 + 0 * 2^7$

$=0+0+0+8+0+32+0+0$
$=40$

7. BITWISE OPERATOR

BITWISE OPERATORS

- There are six types of bitwise operators:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise Left shift operator
5. Bitwise Right shift operator
6. Bitwise complement operator

>> Right shift operator

Shifts bit to the right
a=10, perform a>>1 and a>>2

Convert 10 to binary

2	10	0
2	5	1
2	2	0
2	1	1
	0	

a=10 = 0 0 0 0 1 0 1 0

a>>1

0 0 0 0 1 0 1 0

0 0 0 0 0 1 0 1

0	**0**	**0**	**0**	**0**	**1**	**0**	**1**
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=1 * 2^0 + 0 * 2^1 + 1 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$

$=1+0+4+0+0+0+0+0 = 5$

7. BITWISE OPERATOR

BITWISE OPERATORS

- There are six types of bitwise operators:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise Left shift operator
5. Bitwise Right shift operator
6. Bitwise complement operator

>> Right shift operator

Shifts bit to the right
a=10, perform a>>1 and a>>2

Convert 10 to binary

2	10	0
2	5	1
2	2	0
2	1	1
	0	

a=10 = 0 0 0 0 1 0 1 0

a>>1

0 0 0 0 1 0 1 ~~0~~

0 0 0 0 0 1 0 1

0	0	0	0	0	1	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=1 * 2^0 + 0 * 2^1 + 1 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$

$=1+0+4+0+0+0+0+0 = 5$

a>>2

0 0 0 0 1 0 ~~1~~ ~~0~~

0 0 0 0 0 0 1 0

0	0	0	0	0	0	1	0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$=0 * 2^0 + 1 * 2^1 + 0 * 2^2 + 0 * 2^3 + 0 * 2^4 + 0 * 2^5 + 0 * 2^6 + 0 * 2^7$

$=0+2+0+0+0+0+0+0$

$=2$

7. BITWISE OPERATOR

BITWISE OPERATORS

Find the output of the following code

```
a=10
b=a<<2
c=a>>2
print(b)
print(c)
```

Output:

```
40
2
```

7. BITWISE OPERATOR

BITWISE OPERATORS

- There are six types of bitwise operators:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise Left shift operator
5. Bitwise Right shift operator
6. Bitwise complement operator

~ Ones complement

All 1s are changed to 0s and 0s to 1s

Example:

Find ones complement of 0010

0 0 1 0

1 1 0 1

Example:

Find ones complement of 2

Convert 2 to binary

2	2	0
2	1	1
	0	

2 0 0 0 0 0 0 1 0

~2 1 1 1 1 1 1 0 1

Convert back to decimal

7. BITWISE OPERATOR

```
>>> x=10
>>> y=11
>>> print('~x= ', ~x)
~x=  -11
>>> print('x&y= ', x & y)
x&y=  10
>>> print('x|y= ', x | y)
x|y=  11
>>> print('x^y= ', x ^ y)
x^y=  1
>>> print('x<<2= ', x<<2)
x<<2=  40
>>> print('x>>2= ', x>>2)
x>>2=  2
```

Determine the output of the following program

```
x=3
y=5
print('x&y=', x& y)
print('x|y=', x|y)
print('x^y=', x^y)
print('x left shift 2=', x<<2)
print('x right shift 2=', x>>1)
```

OUTPUT

```
x&y= 1
x|y= 7
x^y= 6
x left shift 2= 12
x right shift 2= 1
```

8. MEMBERSHIP OPERATORS

There are two membership operators.

- in
- not in

in Operator

The in operator returns **True** if the specified **element is present in the sequence**; otherwise, it returns False.

not in Operator

The not in operator returns **True** if the specified element is **not present** in the sequence; otherwise, it returns False.

```
# Example list
fruits = ['apple', 'banana', 'cherry']

# Using 'in' operator
print('apple' in fruits)  # Output: True
print('grape' in fruits)  # Output: False

# Using 'not in' operator
print('grape' not in fruits)  # Output: True
print('banana' not in fruits) # Output: False
```

8. MEMBERSHIP OPERATORS

There are two membership operators.

- in
- not in

If we want to display the members of the list using for loop we use the 'in' operator.

```
names = ["Rani", "Yamini", "Sushmita", "Veena"]

for name in names:
    print (name)
```

Output:

```
Rani
Yamini
Sushmita
Veena
```

When true is returned the print( ) function in the for loop is executed and the name is displayed.

9. IDENTITY OPERATORS

There are two identity operators.

- is
- is not

- These operators **compare the memory locations** of two objects. Hence it is possible to know whether the two objects are same or not.
- **Memory location of an object can be seen using the id( ) function.**

Example:

```
a = 25
b = 25
id(a)
1670954952
id(b)
1670954952
```

Here 25 is the object for which two names are given. If we display identity number we get the same numbers as they refer to the same object.

```
# Example with numbers
a = 25
b = 25

print(a is b)     # Output: True
print(a is not b) # Output: False
```

9. IDENTITY OPERATORS

There are two identity operators.

- is
- is not

1. The is operator

- **The is operator** is used to **compare whether two objects are same or not**.
- It will **internally compare the identity number** of the objects.
- If **identity numbers of the objects are same**, it will return True or else it returns false

2. The is not operator

- Is not operator returns **true** if **identity numbers of two objects being compared are not the same**. If they are same it will return false.

The **is** and **is not** operators **do not compare the values** of the objects.

They **compare** the **identity numbers or memory locations of the objects**.

9. IDENTITY OPERATORS

Program:

```
a = 25
b = 25
if(a is b):
    print("a and b have same identity")
else:
    print("a and b do not have same identity")
```

Output:

```
a and b have same identity
```

Program:

```
one = [1,2,3,4]
two = [1,2,3,4]
if(one is two):
    print("one and two are same")
else:
    print("one and two are not same")
```

Output:

```
one and two are not same
```

'is' operator does not compare the values. It compares the identity numbers of the lists.

```
id(one)
51792432
id(two)
51607192
```

Both lists are stored at different memory locations.

9. IDENTITY OPERATORS

We can use equality operator(==) to compare the values.

```
if(one == two):
    print("one and two are same")
else:
    print("one and two are not same")
```

OUTPUT:

```
one and two are same
```

OPERATOR PRECEDENCE AND ASSOCIATIVITY

Table 4.6: Precedence of Operators in Python

Operator	Name
( )	Parenthesis
**	Exponentiation
-, ~	Unary minus, Bitwise complement
*, / , // , %	Multiplication, Division, Floor division, Modulus
+ , -	Addition, Subtraction
<< , >>	Bitwise left shift, Bitwise right shift
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
>, >=, <, <= , == , !=	Relational (comparison) operators
= , %= , /= , //= , -= , += , *= , **=	Assignment operators
is , is not	Identity operators
in , not in	Membership operators
not	Logical not
or	Logical or
and	Logical and

Associativity

- Associativity is the order in which an expression is evaluated that has multiple operators of same precedence.

Expression	Explanation
value = 3/2*4+3+(10/4)**3-2	
value = 3/2*4+3+2.5**3-2	The expression in ( ) is evaluated first.
value = 3/2*4+3+15.625-2	Exponentiation ** is next.
value = 1.5*4+3+15.625-2 value = 6.0+3+15.625-2	* and / have equal precedence. They are evaluated from left to right (associativity). So, first / and then *.
value = 9.0+15.625-2 value = 24.625-2 value = 22.625	+ and – have equal precedence. They are evaluated from left to right (associativity). So, first + and then -.

MATHEMATICAL FUNCTIONS

- A **module** is a file that **contains group of useful objects like functions, classes or variables**.
- **'math'** is a **module** that **contains several functions to perform mathematical operations**.
- If we **want to use any module** in our python program, first **we should import that module into our program by writing 'import' statement.**
- **To import math module we can write**

Method 1:

```
import math
x=math.sqrt(16)
```

Method 2:

```
import math as m
x=m.sqrt(16)
```

Method 3:

```
from math import sqrt
x=sqrt(16)
```

Method 3:

```
from math import factorial, sqrt
x=sqrt(16)
y=factorial(5)
```

Note: These functions cannot be used with complex numbers. To use functions with complex numbers separate module called 'cmath' is used.

MATHEMATICAL FUNCTIONS

Function	Description
ceil(x)	Raises x value to the next higher integer. If x is integer, then same value is returned. Ex: ceil(4.5) gives 5
floor(x)	Decreases x value to the previous integer value. If x integer, then same value is returned. Ex: floor(4.5) gives 4
degrees(x)	Converts angle value x from radians into degrees. Ex: degrees(3.14159) gives 179.9998479605043
radians(x)	Converts x value from degrees into radians. Ex: radians(180) gives 3.141592653589793
sin(x)	Gives sine value of x. Here x value is given in radians. Ex: sin(0.5) gives 0.479425538604203

MATHEMATICAL FUNCTIONS

Function	Description
cos(x)	Gives cosine value of x. Here x value is given in radians. Ex: cos(0.5) gives 0.8775825618903728
tan(x)	Returns tangent value of x, given x value in radians. Ex: tan(0.5) gives 0.5463024898437905
exp(x)	Returns exponentiation of x. This is same as e **x. Ex: exp(0.5) returns 1.6487212707001282
fabs(x)	Gives the absolute value (or positive quantity) of x. Ex: fabs(-4.56) gives 4.56
factorial(x)	Returns factorial value of x. Raises 'ValueError' if x is not integer or is negative. Ex: factorial(4) gives 24
fmod(x, y)	Returns remainder of division of x and y. fmod() is preferred to calculate modulus for float values than '%' operator. '%' operator works well for integer values. Ex: fmod(14.5, 3) gives 2.5

MATHEMATICAL FUNCTIONS

fsum(values)	Returns accurate sum of floating point values. Ex: fsum([1.5, 2.4, -3.3]) returns 0.6000000000000001
modf(x)	Returns float and integral parts of x. Ex: modf(2.56) gives (0.56, 2.0)
log10(x)	Returns base-10 logarithm of x. Ex: log10(5.2345) gives 0.7188752041406328
log(x, [, base])	Returns the natural logarithm of x of specified base. Ex: log(5.5, 2) gives 2.4594316186372973
sqrt(x)	Returns positive square root value of x. Ex: sqrt(49) gives 7.0
pow(x, y)	Raises x value to the power of y. Ex: pow(5, 3) returns 125.0

MATHEMATICAL FUNCTIONS

Function	Description
gcd(x, y)	Gives greatest common divisor of x and y. Available from Python 3.5 onwards. Ex: gcd(25, 30) gives 5
trunc(x)	The real value of x is truncated to integer value and returned. Ex: trunc(15.5676) gives 15
isinf(x)	Returns True if x is a positive or negative infinity, and False otherwise. Ex: num = float('Inf') # num is a float number that indicates infinity. Isinf(num) gives True
isnan(x)	Returns True if x is a NaN (not a number), and False otherwise. Ex: num = float('NaN') # convert NaN into a float representation. isnan(num) gives True

Table 4.9: Constants in Math Module

Constant	Description
pi	The mathematical constant π = 3.141592..., with high precision.
e	The mathematical constant e = 2.718281..., with high precision.
inf	A floating-point positive infinity. (For negative infinity, use -math.inf.) Equivalent to the output of float('inf')
nan	A floating-point "not a number" (NaN) value. Equivalent to the output of float('nan').

Write a python program to calculate area of a circle.(Use mathematical constant pi)

```
# to calculate area of a circle
import math
r = 15.5
area = math.pi * r ** 2
print("Area of circle= ", area)
```

THANK YOU☺

by PRACHI SURLAKAR